MyInfArith

MD Sameer - CS24BTECH11041

Introduction

The objective of this project is to build a library in Java to perform operations on two very long numbers.

The project contains a main class file `MyInfArith.java` and two files `AInteger.java` and `AFloat.java`.

- `Project/`
  - `MyInfArith.java`
  - `arbitraryarithmetic/`
    - `AInteger.java`
    - `AFloat.java`
  - `build.xml`
  - `MyInfArithmetic.py`

Overview of Project

The project consists of three main classes: `AInteger` and `AFloat`, which handle arbitrary-precision arithmetic operations on integers and floating-point numbers respectively, and `MyInfArith`, which serves as the entry point of the application and coordinates user input with the appropriate arithmetic logic.

AInteger

The class `AInteger` supports arbitrary-precision integer arithmetic. It takes a string as input and converts it into an internal representation using a Arraylist of digits.
It contains datatypes:

- `ArrayList<Integer> digits`
- `boolean isPositive`

AFloat

The class `AFloat` supports arbitrary-precision floating-point arithmetic. It takes a string as input and converts it into an internal representation using a Arraylist of digits.
It contains datatypes:

- `ArrayList<Integer> intnums`
- `ArrayList<Integer> decimlas`
- `boolean isPositive`

MyInfArith

The class `MyInfArith` acts as an interface between the user and the arithmetic classes, using methods from `AInteger` and `AFloat` to perform the desired operations based on user input.

Class Implementation

AInteger

The main methods of this class are:

- `add()`: Performs addition on two arbitrary-length integers, handling both positive and negative values. It uses digit-wise summation with carry propagation. If the operands have opposite signs, the method delegates the operation to the `sub()` method to correctly manage sign and magnitude.
- `sub()`: Performs subtraction on two integers of arbitrary size, correctly handling positive and negative cases. It uses digit-wise subtraction with borrow handling. If the operands have opposite signs, it internally calls the `add()` method to compute the result.
- `mult()`: Performs multiplication of two arbitrary-precision integers. It uses digit-wise multiplication and handles the sign of the product appropriately, based on the signs of the operands.
- `div()`: Performs division between two large integers.
  It computes the quotient using a binary search algorithm. It sets the lower bound to 0 and the upper bound to the dividend (number1). The midpoint is calculated as $\frac{\text{low}+\text{high}}{2}$, and its product with the divisor (number2) is compared with the dividend. If equal, the quotient is found; if less, the lower bound is adjusted; otherwise, the upper bound is reduced. This method ensures accurate results while managing the sign of the quotient correctly.

The following are the additional methods that support the arithmetic operations:

- `toString()` : Converts an `AInteger` object back to String.

- `compare_nums()` : Compares two `AInteger` objects and returns:
  - 1 if the first number is greater,
  - 0 if both are equal,
  - -1 if the first number is smaller.
- `divByTwo()` : Divides an `AInteger` object by 2 using digit-wise division. This method is particularly used in the `div()` method to compute the midpoint during binary search.

AFloat

The main methods of this class are:

- `add()`: Performs addition on two floating-point numbers. It first aligns the decimal parts of both numbers, performs digit-wise addition on the fractional part, then proceeds to the integer part. Carries are handled appropriately across the decimal boundary. When opposite signs the method calls `sub()` method.
- `sub()`: Performs subtraction on two floating-point numbers. It first aligns the decimal parts of both numbers, performs digit-wise subtraction on the fractional part, then proceeds to the integer part. Borrow is handled appropriately across the decimal boundary. When opposite signs the method calls `add()` method.
- `mult()`: Performs multiplication between two floating-point numbers. It copies the `AFloat` objects into `AInteger` representations by removing their decimal points. Then, it performs integer multiplication on these `AInteger` values. Finally, the result is converted back into a floating-point format by splitting the result according to decimal places from both operands.
- `div()`:Performs division between two floating-point numbers with arbitrary precision. It copies both numbers into `AInteger()` representations—removing the decimal points. To have required precision 30 trailing zeroes are appended to the numerator then, the division is handled using the `AInteger` class. After computing the integer result, the decimal point is reinserted.

The following are the additional methods that support the arithmetic operations:

- `toString()` : Converts an `AFloat` object back to its string representation with the decimal point correctly positioned.
- `compare_nums()` : Compares two `AFloat` objects by aligning their decimal points and performing a digit-wise comparison:
  - 1 if the first number is greater,
  - 0 if both numbers are equal,
  - -1 if the first number is smaller.

Usage of Library

Command Line Interface

The project provides a command-line interface through the `MyInfArith.java` class. This class parses the input arguments and invokes the appropriate methods in either the `AInteger` or `AFloat` classes.

Usage Pattern:

```
java MyInfArith <int/float> <add/sub/mult/div> <number1> <number2>
```

Examples:

- `java MyInfArith int add 164884844 -6846464646666`
  Output: The Answer is -6681579801822
- `java MyInfArith float div 444634341544.13134 65633435434`
  Output: The Answer is 6.774509647468461051881375757393817 97
- `java MyInfArith float sub 5.9585 -2.2565`
  Output: The Answer is 8.2150

MyInfArith.py

The Python script `MyInfArith.py` automates the compilation and execution process. It uses Apache Ant to compile the Java files and then executes the program with the provided input arguments.

Usage:

```
python MyInfArith.py <int/float> <add/sub/mult/div> <number1> <number2>
```

build.xml

The `build.xml` file is the Ant build script that defines multiple targets for building and managing the project.

- **init:** Creates the `build` directory to store compiled class files.
- **clean:** Deletes all compiled files from the `build` directory.
- **compile:** Compiles all source files and outputs the class files into the `build` directory.
- **run:** Executes the program using input arguments passed through Ant properties.
- **jar:** Packages the compiled classes into a `.jar` file.

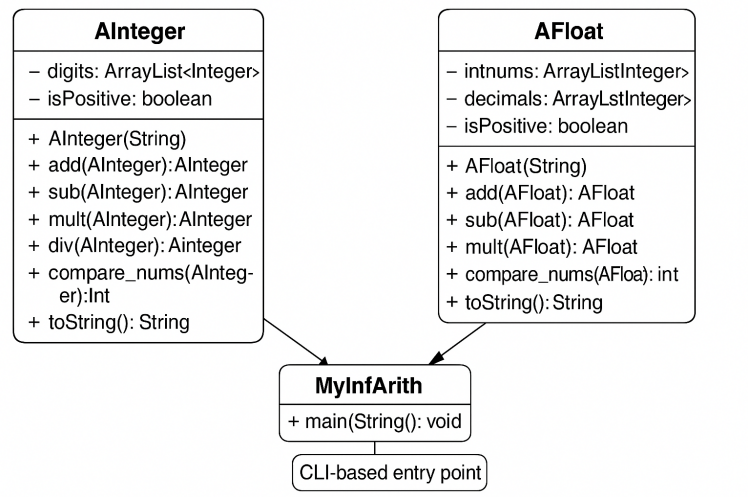

Figure 1: UML Diagram

Conclusion

This arbitrary-precision arithmetic library provides a better solution for performing large number arithmetic operations in Java. By using string manipulation to handle large numbers, the library can support integers and floating-point numbers of arbitrary size.